DataSet here is a 1-Channel EEG Data collected from 500 people with Epilepsy with and without seizure along with healthy people for about 23.6 seconds. The goal is to make a ML Classifier to be able to seperate and identify them.

Data PreProcessing

Reading Data from text files and turning them into .mat files to be able to manipulate them super fast. And the designing a band-stop notch filter for 50Hz powerline noise removal.

```
% Reading Raw Data
A = zeros(100, 4097);
B = zeros(100, 4097);
C = zeros(100, 4097);
D = zeros(100, 4097);
E = zeros(100, 4097);

% Read Data From Text
for it = 1:100

    if (it<10)
        A(it, :) = textread(['RawDataset\Z\Z00' num2str(it) '.txt'], '%d');
        B(it, :) = textread(['RawDataset\O\O00' num2str(it) '.txt'], '%d');
        C(it, :) = textread(['RawDataset\N\N00' num2str(it) '.txt'], '%d');
        D(it, :) = textread(['RawDataset\F\F00' num2str(it) '.txt'], '%d');
        E(it, :) = textread(['RawDataset\S\S00' num2str(it) '.txt'], '%d');
    elseif it >= 10 && it < 100
        A(it, :) = textread(['RawDataset\Z\Z0' num2str(it) '.txt'], '%d');
        B(it, :) = textread(['RawDataset\O\O0' num2str(it) '.txt'], '%d');
        C(it, :) = textread(['RawDataset\N\N0' num2str(it) '.txt'], '%d');
        D(it, :) = textread(['RawDataset\F\F0' num2str(it) '.txt'], '%d');
        E(it, :) = textread(['RawDataset\S\S0' num2str(it) '.txt'], '%d');
    elseif it == 100
        A(it, :) = textread(['RawDataset\Z\Z' num2str(it) '.txt'], '%d');
        B(it, :) = textread(['RawDataset\O\O' num2str(it) '.txt'], '%d');
        C(it, :) = textread(['RawDataset\N\N' num2str(it) '.txt'], '%d');
        D(it, :) = textread(['RawDataset\F\F' num2str(it) '.txt'], '%d');
        E(it, :) = textread(['RawDataset\S\S' num2str(it) '.txt'], '%d');
    end
end

% Designing Butterworth Notch Filter for 50Hz Noise Removal
fs = 173.61;                        % Sampling Freq
wl = 49;                            % Cuoff LowerBound
wh = 51;                            % Cuoff UpperBound
order = 3;                          % Filter Order
type = 'stop';                      % Filter Type

[b, a] = butter(order,  [wl,  wh]/(fs/2),  type);           % Create Filter

% Apply Filter
for it = 1:size(A, 1)

    A(it, :) = filtfilt(b, a, A(it, :));
```

```
    B(it, :) = filtfilt(b, a, B(it, :));
    C(it, :) = filtfilt(b, a, C(it, :));
    D(it, :) = filtfilt(b, a, D(it, :));
    E(it, :) = filtfilt(b, a, E(it, :));

end

% Saving Prepared Data
RawData.Fs = fs;                   % Sampling Frequency

% Saving EEGs And Labels of'em
RawData.A = A;
DataLabels.A = repmat({'Normal'}, size(A, 1), 1);
RawData.B = B;
DataLabels.B = repmat({'Normal'}, size(B, 1), 1);
RawData.C = C;
DataLabels.C = repmat({'EpiNoSeiz'}, size(C, 1), 1);
RawData.D = D;
DataLabels.D = repmat({'EpiNoSeiz'}, size(D, 1), 1);
RawData.E = E;
DataLabels.E = repmat({'EpiSeiz'}, size(E, 1), 1);

save RawData RawData
save DataLabels DataLabels

disp('EEG Read and Saved as "RawData.mat".');
```

```
EEG Read and Saved as "RawData.mat".
```

```
clear;
```

Feature Extraction

This section is responsible for seperating delta, theta, alpha, beta and gamma waves and Extract time and frequency domain features and saving them.

Time Domain

```
% Load File,  Parameter Daclaration & Initialization
load RawData

Fs = RawData.Fs;          % Sampling Frequency
A = RawData.A;
B = RawData.B;
C = RawData.C;
D = RawData.D;
E = RawData.E;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Rhythm Filtering %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Each Column Contain Frequency Bounds of Delta,  Theta,
% Alpha,  Beta and Gamma Waves Respectively..
```

```
bound = [0.1,  4,  8,   12,  30
    4,  7,  12,  30,  70];

% Filter Options
type = 'bandpass';            % Filter Type
order = 3;                    % Filter Order

% Designing Filters,  Frequency Seperation and Time Feature Extraction
for it = 1:size(A, 1)

    for freqIt = 1:size(bound, 2)

        % Seperate Rhythms and Perform Feature Selection
        [b, a] = butter(order, [bound(:, freqIt)']/(Fs/2), type);

        tempA = filtfilt(b, a, A(it, :));
        tempB = filtfilt(b, a, B(it, :));
        tempC = filtfilt(b, a, C(it, :));
        tempD = filtfilt(b, a, D(it, :));
        tempE = filtfilt(b, a, E(it, :));

        tempFA(freqIt, :) = MyFeatureExtractor(tempA);
        tempFB(freqIt, :) = MyFeatureExtractor(tempB);
        tempFC(freqIt, :) = MyFeatureExtractor(tempC);
        tempFD(freqIt, :) = MyFeatureExtractor(tempD);
        tempFE(freqIt, :) = MyFeatureExtractor(tempE);

    end

    TimeFeatures.A(it, :) = reshape(tempFA', 1, size(bound, 2)*8);
    TimeFeatures.B(it, :) = reshape(tempFB', 1, size(bound, 2)*8);
    TimeFeatures.C(it, :) = reshape(tempFC', 1, size(bound, 2)*8);
    TimeFeatures.D(it, :) = reshape(tempFD', 1, size(bound, 2)*8);
    TimeFeatures.E(it, :) = reshape(tempFE', 1, size(bound, 2)*8);

end

% Save Again
save TimeFeatures TimeFeatures

disp("Time Domain Features Extracted and Saved into TimeFeatures.mat .")
```

```
Time Domain Features Extracted and Saved into TimeFeatures.mat .
```

```
clear;
```

Frequency Domain

```
% Load File,  Parameter Daclaration & Initialization
load RawData

Fs = RawData.Fs;          % Sampling Freq
A = RawData.A;
B = RawData.B;
```

```
C = RawData.C;
D = RawData.D;
E = RawData.E;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Rhythm Filtering %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Each Column Contain Frequency Bounds of Delta,  Theta,
% Alpha,  Beta and Gamma Waves Respectively..

bound = [0.1,  4,  8,   12,  30
    4,  7,  12,  30,  70];

%% Freq Seperation and Freq Feature Extraction
siglen = size(A, 2);
x = linspace(0, Fs/2, ceil(siglen/2));

for it = 1:size(A, 1)

    fft_A = fft(A(it, :));
    fft_B = fft(B(it, :));
    fft_C = fft(C(it, :));
    fft_D = fft(D(it, :));
    fft_E = fft(E(it, :));

    fft_A = abs(fft_A(1:ceil(siglen/2)));
    fft_B = abs(fft_B(1:ceil(siglen/2)));
    fft_C = abs(fft_C(1:ceil(siglen/2)));
    fft_D = abs(fft_D(1:ceil(siglen/2)));
    fft_E = abs(fft_E(1:ceil(siglen/2)));

    % Seperate Rhythms and Perform Feature Selection
    for freqIt = 1:size(bound, 2)

        index = find(x >= bound(1, freqIt) & x <= bound(2, freqIt));

        tempFA(freqIt, :) = MyFeatureExtractor(fft_A(index));
        tempFB(freqIt, :) = MyFeatureExtractor(fft_B(index));
        tempFC(freqIt, :) = MyFeatureExtractor(fft_C(index));
        tempFD(freqIt, :) = MyFeatureExtractor(fft_D(index));
        tempFE(freqIt, :) = MyFeatureExtractor(fft_E(index));

    end

    FrequencyFeatures.A(it, :) = reshape(tempFA', 1, size(bound, 2)*8);
    FrequencyFeatures.B(it, :) = reshape(tempFB', 1, size(bound, 2)*8);
    FrequencyFeatures.C(it, :) = reshape(tempFC', 1, size(bound, 2)*8);
    FrequencyFeatures.D(it, :) = reshape(tempFD', 1, size(bound, 2)*8);
    FrequencyFeatures.E(it, :) = reshape(tempFE', 1, size(bound, 2)*8);

end

% Save Again
save FreqFeatures FrequencyFeatures

disp("Frequency Domain Features Extracted and Saved into FreqFeatures.mat .")
```

```
Frequency Domain Features Extracted and Saved into FreqFeatures.mat .
```

```
clear;
```

Feature Concatenation and Preparation

```
load FreqFeatures
load TimeFeatures
load DataLabels

AFeatures = [FrequencyFeatures.A, TimeFeatures.A];
BFeatures = [FrequencyFeatures.B, TimeFeatures.B];
CFeatures = [FrequencyFeatures.C, TimeFeatures.C];
DFeatures = [FrequencyFeatures.D, TimeFeatures.D];
EFeatures = [FrequencyFeatures.E, TimeFeatures.E];

Features = [AFeatures
    BFeatures
    CFeatures
    DFeatures
    EFeatures];

Labels = [DataLabels.A
    DataLabels.B
    DataLabels.C
    DataLabels.D
    DataLabels.E];

save Final Features Labels

disp("Features Concatenated and Saved as Final.mat .")
```

```
Features Concatenated and Saved as Final.mat .
```

```
clear;
```

Main Classification

Here as the final and main section of the code, collective feature selection for the extracted feature are performed with the use of Bio-Geography-Based optimization algorithm. and Classifying with KNN and LDA and Decision Tree.

There are bunch of options that can be selected and changed. Like Mutaion Rate, Feature Num, Classifier type, CostFunction...

There will be Final File saved as `'Soltion.mat'`. Which contains the found Solution and the InterClass Accuracy of the Data.

The process is Accelerated Using Parallel Prgramming Toolbox!

```
% Loading Datas with Labels and Features
load Final
```

```
% Starting Parallel Pool
p = parpool('local', 4);
```

```
Starting parallel pool (parpool) using the 'local' profile ...
Connected to the parallel pool (number of workers: 4).
```

```
% BBO Parameters
Range = [1, size(Features, 2)];
MaxIt = 100;
HabitationNum = 50;
CostFunction = @(x) Classifier(Features, Labels, x);

% See if We Wanna reduce Numeber of Features to n What Would They Be?
FeatureNum = [5, 10, 15, 20, 25, 30];

% BBO Evaluation for Feature Reduction
for iter = FeatureNum
    % Store the Solutions
    Solutions(iter) = DiscreteBBO(CostFunction, HabitationNum, MaxIt, iter, Range);
end

% ShutDown the Current Pool
delete(gcp('nocreate'))

save Solutions Solutions
disp('Totally Done! Solutions Saved in Solutions.mat .')
```

```
Totally Done! Solutions Saved in Solutions.mat .
```

```
clear;
```